

Hibernate

- Slides based on Gavin Kings presentation at JA00 2003

Hibernate

- Relational Persistence for Java and .NET
- Opensource (LGPL)
- Mature and popular
- Custom API



Auction Object Model

AuctionItem	1	*	Bid
description type	item	bids	amount datetime
		0..1	
	successfulBid		

Persistent Class

- Default constructor
- Get/set pairs
- Collection property is an interface type
- Identifier property

```
public class AuctionItem {  
    private Long _id;  
    private Set _bids;  
    private Bid _successfulBid  
    private String _description;  
  
    public Long getId() {  
        return _id;  
    }  
    private void setId(Long id) {  
        _id = id;  
    }  
    public String getDescription() {  
        return _description;  
    }  
    public void setDescription(String desc) {  
        _description=desc;  
    }  
    ...  
}
```

XML Mapping

- Readable metadata
- Column / table mappings
- Surrogate key generation strategy
- Collection metadata
- Fetching strategies

```
<class name="AuctionItem" table="AUCTION_ITEM">
  <id name="id" column="ITEM_ID">
    <generator class="native"/>
  </id>
  <property name="description" column="DESCR"/>
  <many-to-one name="successfulBid"
    column="SUCCESSFUL_BID_ID"/>
  <set name="bids"
    cascade="all"
    lazy="true">
    <key column="ITEM_ID"/>
    <one-to-many class="Bid"/>
  </set>
</class>
```

Dirty Checking

Retrieve an `AuctionItem` and change description

```
Session session = sessionFactory.openSession();
Transaction tx = s.beginTransaction();

AuctionItem item =
    (AuctionItem) session.get(AuctionItem.class, itemId);
item.setDescription(newDescription);

tx.commit();
session.close();
```

Transitive Persistence

Retrieve an `AuctionItem` and create a new persistent `Bid`

```
Bid bid = new Bid();  
bid.setAmount(bidAmount);  
  
Session session = sf.openSession();  
Transaction tx = session.beginTransaction();  
  
AuctionItem item =  
    (AuctionItem) session.get(AuctionItem.class, itemId);  
bid.setItem(item);  
item.getBids().add(bid);  
  
tx.commit();  
session.close();
```

Detachment

Retrieve an `AuctionItem` and create a new persistent Bid

```
Session session = sf.openSession();  
Transaction tx = session.beginTransaction();  
AuctionItem item =  
    (AuctionItem) session.get(AuctionItem.class, itemId);  
tx.commit();  
session.close();
```

```
item.setDescription(newDescription);
```

```
Session session2 = sf.openSession();  
Transaction tx = session2.beginTransaction();  
session2.update(item);  
tx.commit();  
session2.close();
```


Transparent Lazy Fetching

```
AuctionItem item = (AuctionItem) session.get(AuctionItem.class, itemId);
```

```
    SELECT ... FROM AUCTION_ITEM ITEM WHERE ITEM.ITEM_ID = ?
```

```
Iterator iter = item.getBids().iterate();
```

```
    SELECT ... FROM BID BID WHERE BID.ITEM_ID = ?
```

```
item.getSuccessfulBid().getAmount();
```

```
    SELECT ... FROM BID BID WHERE BID.BID_ID = ?
```

Hibernate Query Options

- Hibernate Query Language (HQL)
 - “Minimal” OO dialect of ANSI SQL
- Criteria Queries
 - Extensible framework for expressing query criteria as objects
 - Includes “query by example”
- Native SQL Queries

Hibernate Query Language

- Make SQL be object oriented
 - Classes and properties instead of tables and columns
 - Polymorphism
 - Associations
 - *Much* less verbose than SQL
- Full support for relational operations
 - Inner/outer/full joins, cartesian products
 - Projection
 - Aggregation (max, avg) and grouping
 - Ordering
 - Subqueries
 - SQL function calls

Hibernate Query Language

- HQL is a language for talking about “sets of objects”
- It unifies *relational operations* with *object models*

Hibernate Query Language

Simplest HQL Query:

`from AuctionItem`

i.e. get all the AuctionItems:

```
List allAuctions = session.createQuery("from AuctionItem")  
    .list();
```

Hibernate Query Language

More realistic example:

```
select item
from AuctionItem item
    join item.bids bid
where item.description like 'hib%'
    and bid.amount > 100
```

i.e. get all the **AuctionItems** with a **Bid** worth > 100 and description that begins with "hib"

Hibernate Query Language

Projection:

```
select item.description, bid.amount  
from AuctionItem item  
    join item.bids bid  
where bid.amount > 100  
order by bid.amount desc
```

i.e. get the description and amount for all the **AuctionItems**
with a **Bid** worth > 100

Hibernate Query Language

Aggregation:

```
select max (bid.amount) , count (bid)
from AuctionItem item
    left join item.bids bid
group by item.type
order by max (bid.amount)
```


Hibernate Info

- <http://hibernate.org>
- Hibernate in Action (Manning, 2004)
- Tool support
 - <http://xdoclet.sf.net>
 - <http://boss.bekk.no/boss/middlegen>
 - <http://www.andromda.org/>
 - <http://www.hibernate.org/255.html>

Misc

- Be sure to select the right strategy for auto generating the primary key
- Automatically recreating database from schema does not work if new schema violates old foreign key constraints